CacheLab: Enhancing Memory Hierarchy Learning Through Interactive Exploration

Hugo Jeller Ferreira
FACOM - UFMS
Campo Grande, Brazil
hugo.jeller@ufms.br

Vinícius Shinohara
FACOM - UFMS
Campo Grande, Brazil
v.shinohara@ufms.br

Gregório Koslinksi
FACOM - UFMS
Campo Grande, Brazil
gregorio.neto@ufms.br

Liana Duenha
FACOM - UFMS
Campo Grande, Brazil
liana.duenha@ufms.br

***Abstract*—This research-to-practice category full paper presents CacheLab, an open-source, web-based cache simulator, and reports a case study evaluating its application in undergraduate and graduate courses at the Federal University of Mato Grosso do Sul, Brazil. Designed to support both education and research, CacheLab provides an intuitive simulation environment for exploring cache mechanisms and advanced features for performance analysis. The tool was integrated into the Computer Architecture course for Computer Science and Engineering students, aiming to enhance the teaching of memory hierarchy concepts. The case study involved a four-hour session combining a theoretical overview of cache design with guided experimental exercises using CacheLab. Students' knowledge was assessed through pre- and post-activity questionnaires and technical reports based on practical exercises. The results suggest that simulation-based tools like CacheLab can promote significant conceptual learning, even with limited prior instruction. The evaluation also applied the Technology Acceptance Model (TAM) to analyze students' perceptions of the tool. Findings indicate high levels of acceptance, with CacheLab demonstrating strong usability and Perceived Usefulness, in line with user-centered design principles in educational technologies.**

***Index Terms*—Cache memory simulation; Computer Architecture education; Technology Acceptance Model (TAM); Web-based tools.**

I. Introduction

The disparity between processing speeds and memory latency has emerged as a critical bottleneck in the evolution of modern computer systems. In the 1960s, cache memories were introduced to address this challenge, using advanced technologies to enhance the efficiency of data exchange between processors and storage systems.

Over time, cache memories have become widespread and more cost-effective, establishing themselves as indispensable components of data processing systems, from high-performance servers to low-capacity embedded processors. Modern personal computers and mobile devices typically incorporate multiple levels of cache, with capacities ranging from a few megabytes to tens of gigabytes [1], [2].

In the field of Computer Architecture within undergraduate computing courses (e.g., Computer Science, Computer Engineering, Information Systems, and Software Engineering), topics such as cache design models, write policies, replacement strategies, and memory performance metrics are commonly covered. Professors often use graphical representations to reinforce these concepts and simulate cache operations at each memory access. However, these representations are typically large and time-consuming, requiring significant class time. Additionally, exploring the impact of a cache design on system performance requires effectively analyzing many cache accesses. For example, many cache misses are expected to occur during the first cache accesses, which can lead to a very high miss rate. If cache performance is evaluated after only a few accesses, it can create the mistaken impression that the cache design is inefficient. This task is challenging to accomplish using a whiteboard or even slide presentations.

To address this limitation, educators often rely on memory simulators as tools to support the teaching and learning process. Cache simulators can be categorized into two types: those used for educational purposes and those designed for performance evaluation. Educational simulators are typically simplistic, using fictitious or randomly generated memory traces that do not reflect the behavior of real applications [3], [4]. In contrast, performance evaluation simulators are more complex but often lack user-friendly interfaces, making them less suitable for educational contexts [5].

This paper introduces CacheLab, a functional cache memory simulator designed to support the teaching of Computer Architecture and related disciplines in Computer Science courses. A case study applying it in undergraduate and graduate courses is also presented. CacheLab effectively bridges the gap between research-oriented and educational simulators by offering an intuitive interface alongside robust features that address key cache memory concepts while enabling the evaluation of memory system performance. The tool was evaluated in the Computer Architecture course within both Computer Science and Engineering programs, and student feedback was gathered to assess its effectiveness in teaching memory system design and performance analysis.

This paper is organized as follows. Section II provides an overview of cache simulators, highlighting their main features. Section III describes the main features of CacheLab. Section IV explains the methodology applied in the case study. Section V presents the results obtained, along with their analysis and discussion. Section VI outlines the findings of this work and suggests directions for future research.[1]

[1]This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001.

II. Related Work

This section presents cache memory simulators identified in the literature that share educational purposes and functionalities with CacheLab. These tools support the teaching of computer architecture, enabling the configuration of cache parameters and the visualization of memory operations. The following subsections summarize the main features and limitations of these simulators.

Beto [3] is a cache memory simulator named in honor of Dr. Robert H. Dennard, the inventor of DRAM. It offers an accessible environment for exploring cache memory operation, featuring a user-friendly interface with color coding and animations to visualize cache access. Its step-by-step monitoring functionality enhances the educational experience by allowing users to closely follow memory operations. However, a major limitation is the requirement to manually enter memory addresses as integers, restricting the exploration of real-world memory access patterns and limiting advanced analyses and its applicability to modern systems.

The University of California Simulator [4] provides a learning platform for modeling memory hierarchies and evaluating cache performance. It allows users to configure basic cache parameters and provides statistics such as hit rate and average access time. Nevertheless, its fixed two-way set-associative configuration and functional issues, like the inability to easily remove cache blocks, reduce its flexibility. Moreover, requiring manual entry of memory addresses further positions it as a tool more suitable for beginners.

The University of Washington includes the CSE351 cache simulator in its instructional materials to support learning in the Hardware and Software Interface course [6]. This simulator offers system configuration options such as address size, cache size, block size, associativity, write policy, and replacement method. The platform displays a history of accesses and messages detailing the processes to aid in visualizing the simulation and providing explanations. Like the simulators presented previously, the University of Washington simulator requires addresses provided manually by the user in hexadecimal format.

The simulators discussed in this section share functionalities similar to CacheLab and are designed to support the teaching of computer architecture in undergraduate computer science courses. They also facilitate cache design through parameter configuration.

However, a key distinction lies in the input data. The presented simulators require manually generated memory requests, which fail to reflect the principles of locality (temporal and spatial) or demand individually loaded inputs, making them even less suitable for evaluating cache designs.

In contrast, CacheLab enables the simulation of cache designs using memory traces derived from real applications. This approach is more realistic and suitable for robust performance evaluation.

III. The CacheLab

This simulator's primary purpose is to provide a detailed analysis of the behavior and performance of different cache-memory organizations. This analysis utilizes memory access patterns instrumented from real applications, enabling the exploration of the principles of spatial and temporal locality. The available functionalities and the user interface enable the simulator to serve as a didactic tool for teaching Computer Architecture.

The simulator will be described step by step. Section III-A presents the main features for cache designs; Section III-B describes how to generate the memory traces executed by the simulator, and Section III-C outlines the key technologies for implementation and the optimizations adopted.

A. Features and Contributions

The key features and contributions of this simulator include:

- **Realistic Input Data:** Enables the evaluation of memory *traces* obtained from real applications, avoiding reliance on random or synthetic address simulations.
- **Customizable Cache Design:** Allows users to configure set-associative caches with one or multiple levels, customizing parameters such as block size, number of blocks, associativity, and replacement policies for each level.
- **Configurable Performance Metrics:** Supports the definition of performance parameters, including *hit time* and *miss penalty*, enabling detailed performance analysis.
- **Flexible Simulation Modes:** Offers both access-by-access simulation—ideal for educational purposes—and full execution simulations, better suited for performance analysis and benchmarking of different cache architectures.
- **Optimized Resource Utilization:** Implements highly efficient processing techniques to minimize resource consumption, allowing the simulation of caches with virtually unlimited sizes.
- **User-Friendly and Accessible:** Provides an intuitive web-based interface, eliminating the need for complex installations and dependencies on specific compilers, operating systems, or advanced technical expertise.

Figure 1 shows the home screen of the CacheLab. The interface includes a top navigation bar with several drop-down menus and control buttons. The simulator usage is structured into three main steps: configuring the cache, selecting a program, and controlling the simulation. The first button is used to select existing cache designs. The second button is used to create a new cache design (detailed in Figure 2a) and to edit or delete the selected one. The interface enables users to configure key cache parameters using sliders and radio buttons, including the number of sets, blocks per set, and words per block. Additionally, the user can select a block replacement policy from LRU (Least Recently Used), FIFO (First In, First Out), and Random. There is also an option to define cache timing parameters, precisely *hit time* and

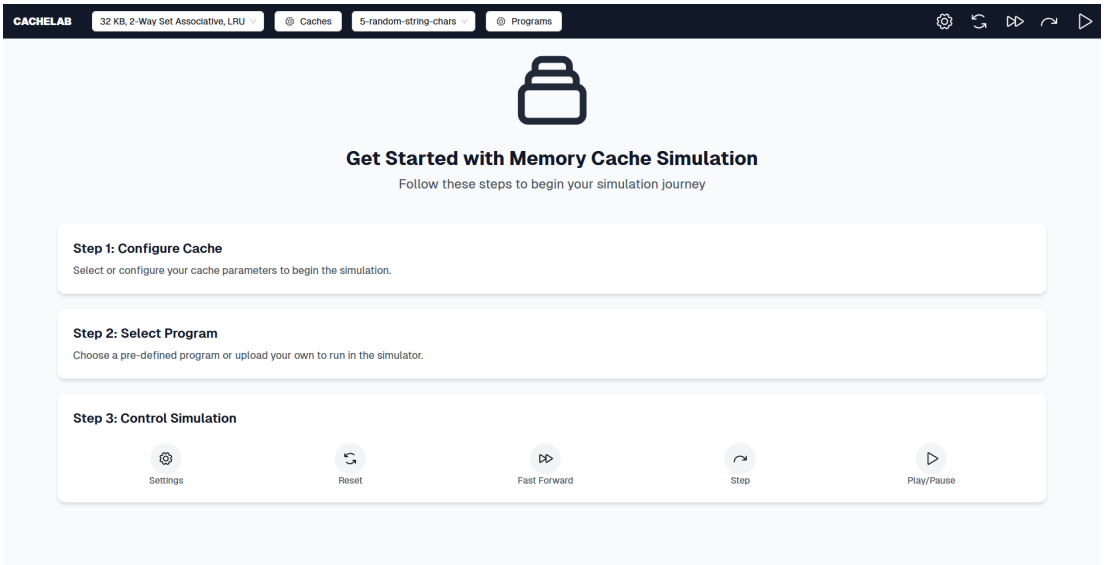
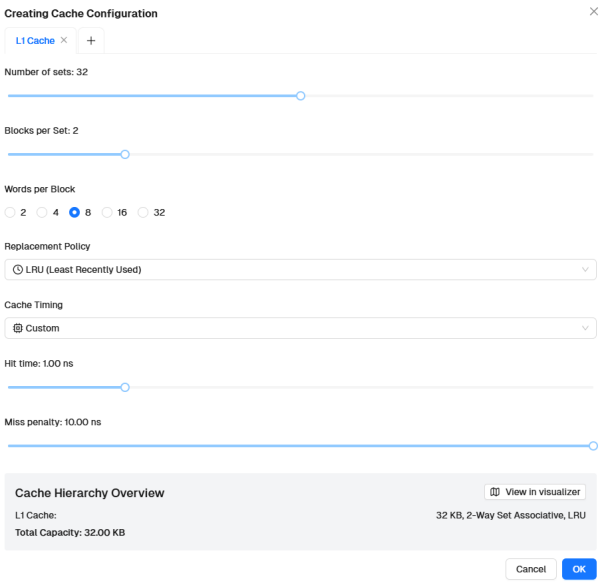
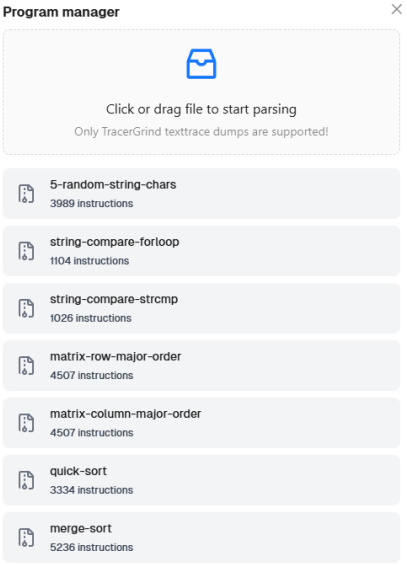


Fig. 1: CacheLab Home.



(a) Cache Design Parameters.



(b) Program Manager.

Fig. 2: Steps 1 and 2 - Cache Design Parameters and Program Manager.

miss penalty, measured in nanoseconds, which are crucial for performance analysis. At the bottom, a summary section provides an overview of the cache hierarchy, displaying the total capacity and associativity. Similarly, users can configure additional cache levels, with each level being parameterized and having its simulation statistics displayed in a dedicated tab within the simulator interface. The third button selects the file that will have its memory trace in the simulation (Figure 2b). There are seven preexisting options already within the project that allow tool testing, with their names referencing the program function. Each contains the trace of all the addresses used in memory references by the program. The fourth button is for uploading a JSON (JavaScript Object Notation) file containing a sequence of hexadecimal memory addresses.

The nomenclature adopted for cache design and performance parameters follows the conventions presented in textbooks widely used for teaching computer architecture in top universities worldwide [1], [2].

Figure 3 shows the CacheLab simulation screen, which is divided into three main sections: cache state, details of the last executed operation, and a history log. The cache state section displays each block individually, grouped by associativity sets. Users can view the address range loaded in each block, the

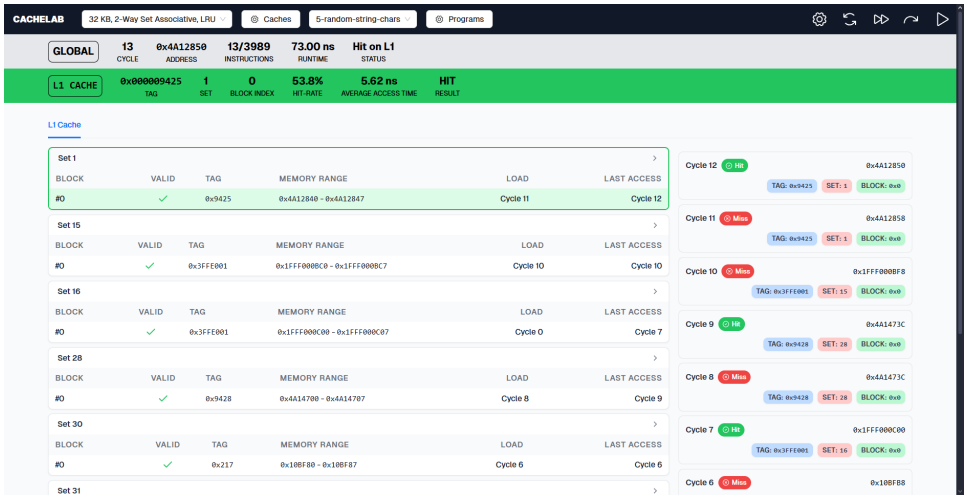


Fig. 3: Cache Simulation.

cycle when the data was loaded, and the last cycle it was accessed. During execution, these blocks are highlighted if accessed in the previous operation.

The details of the last operation section display the accessed address for each cache level, including the calculated *tag*, set index, and the block selected according to the configured replacement policy. The performance metrics, such as *hit rate*, average memory time, and the number of accesses, enable users to monitor the cache during execution.

Finally, the history section lists recently accessed addresses, allowing for an analysis of which sets and blocks are most frequently utilized and identifying addresses that have caused misses.

B. Memory Traces

A memory trace is a sequence of all events that involve memory accesses during the execution of a program. These events include information such as the addresses accessed, the operation mode (read or write), the data being transferred, and the address of the instruction that triggered the operation. The memory traces available are:

- **5-random-string-chars**: This program allocates a five-character string, fills it by looping five times over a function that retrieves a random character from the word `potato`, and then prints the result.
- **string-compare-forloop**: This program allocates memory for a copy of a string, then compares the original with its copy through a custom-implemented string compare function.
- **quick_sort**: Sorts a set of elements using the divide-and-conquer strategy.
- **merge_sort**: It sorts a set of elements using the divide-and-conquer strategy, by repeatedly dividing the dataset into smaller sublists, sorting them, and then merging the sorted sublists back together.
- **matrix-row-major-order**: Dynamically allocates a matrix, initializes it with random values, and searches for the largest element by traversing the matrix row by row.
- **matrix-column-major-order**: Dynamically allocates a matrix, initializes it with random values, and searches for the largest element by traversing the matrix column by column.

To evaluate caches with custom memory traces, users must generate them offline and import them into CacheLab. The process begins by selecting a target application and collecting its memory access patterns. These traces must then be processed into a structured format compatible with the simulator, containing only the memory accesses relevant to the analysis. Finally, the processed trace is imported into CacheLab, and the cache parameters are configured for simulation. These steps are detailed in the following.

To generate a memory trace for a new program, we utilized TracerGrind [7], a tool built on Valgrind [8] that provides comprehensive instrumentation of executed instructions, supporting X86, X86-64, and ARM architectures. TracerGrind captures memory access patterns in real time, logging crucial details such as accessed addresses, operation modes (read or write), and the instructions that cause each access. This data is stored in a binary file, exemplified in the Listing 1.

Listing 1: Execution log

```
[I] 00000000040210da: cmp rax, 0x22
[I] 00000000040210de: jbe 0x40210c9
[M] EXEC_ID:3 INS_ADDRESS:00000000040210c9
START_ADDRESS:000000000403aba0
LENGTH:8 MODE:W DATA:809e030400000000
[M] EXEC_ID:3 INS_ADDRESS:00000000040210cd
START_ADDRESS:0000000004039e90
LENGTH:8 MODE:R DATA:0400000000000000
[B] EXEC_ID:3 THREAD_ID:0000000000000001
START_ADDRESS:00000000040210c9
END_ADDRESS:00000000040210de
```

In this log, a machine instruction (indicated by [I]) starts with the memory address from which it is loaded,

followed by the corresponding mnemonic of the instruction, along with the executed registers and immediate values. Executable blocks (indicated by [B]) have a unique identifier (`EXEC_ID`) associated with the thread identifier (`THREAD_ID`) in which the execution occurred. The `START_ADDRESS` and `END_ADDRESS` fields indicate the first and last executed instructions within that block. Next, the following information is provided about (indicated by [M]): the base memory address (`START_ADDRESS`), the number of bytes accessed (`LENGTH`), the access mode (`MODE`), and the accessed data (`DATA`), representing the value to be written when `MODE` = `W` and the value read when `MODE` = `R`).

The `readelf` tool extracts all stored data from a Linux executable binary. Among the retrieved data are the addresses of the code segment (`.text`) and the symbol table (`.symtab`), which contains the addresses of functions referenced during execution, including the `main` function. Then, it is possible to calculate the relative address of the `main` function within the binary file, allowing execution and termination blocks to be identified during runtime, regardless of the base address chosen by the operating system to load instructions into memory. This approach enables filtering memory accesses caused exclusively by the code of interest to the user.

Separately from the simulator, an auxiliary tool was also developed for processing and validating memory access traces: TracerGrind-Parser [9]. This tool is responsible for converting the text-based trace into a structured object, enabling the identification of relevant execution blocks. Additionally, it processes SIMD (Single Instruction, Multiple Data) instructions by breaking them down into a sequence of word-sized accesses, which, in the simulator implementation, is fixed at 64 bits.

The tool calculates relative addresses for the start and end of the `main` function and uses them as reference points for addresses reported by TracerGrind during application instrumentation. It identifies all execution blocks between the start block, whose first instruction has a relative address equal to the `main` function's address, and the end block, whose last instruction has a relative address equal to the `main` function's address plus its size. As output, the tool generates a JSON-formatted file structured for compatibility with the simulator, allowing seamless integration through its graphical interface.

C. Technologies for Developing and Optimizing the Simulator

The simulator was developed as a *web* application to ensure easy accessibility for undergraduate students, eliminating installation and update requirements while ensuring compatibility across operating systems. The design is entirely *client-side*, avoiding the need for server infrastructure and enabling cost-free hosting through platforms like GitHub Pages.

The *Vite* [10] tool was used to optimize the development workflow, while unit tests were implemented with *Vitest* [11] for seamless and efficient testing. The interface leverages *Ant Design* [12], a library of accessible, pre-built components, ensuring consistency and adherence to accessibility standards. Additionally, *TailwindCSS* [13] was used to streamline the creation of customizable layouts and support rapid prototyping.

Allowing the processing of thousands of memory accesses required several optimizations to ensure that the simulator's performance did not discourage the user from running it thoroughly to evaluate the final performance of the cache system. Therefore, various optimizations were implemented throughout the tool's development to minimize computational resource usage during simulations and guarantee that the simulator runs accordingly on any device capable of supporting a modern web browser.

To significantly reduce memory allocation, the simulator was designed not to store the data accessed by the cache. Instead, it maintains only references to the memory addresses currently loaded. A *LazyArray* structure was introduced for further memory allocation optimizations, enabling the development of components without requiring allocation during initialization. As a result, for high-capacity cache simulations, sets or blocks that are not accessed consume no resources. In addition, these optimizations ensure that initializing a cache of arbitrary size always occurs in constant time.

Beyond memory optimization, access addresses are precomputed to minimize overhead during simulation execution. The cache access history is limited to prevent excessive memory consumption, and several measures were taken to reduce redundant interface updates. Code segments with high-performance impact potential were monitored after each update to avoid performance regressions.

Simulators developed in JavaScript are subject to the limitations of the IEEE 754 standard, which limits integers to 53 bits of precision. This restriction prevents the simulators mentioned here from adequately processing memory traces of real applications, which explore the entire 64-bit address space. CacheLab addressed this limitation by extensively using the BigInt type, which can represent integers with arbitrary precision.

IV. Case Study

This section presents a case study to evaluate the CacheLab Simulator in the context of teaching and learning cache memory hierarchy in undergraduate Computer Science and Computer Engineering courses. This case study was motivated by the following questions:

1) Is CacheLab a useful and user-friendly educational resource for teaching 'Cache Memory Design and Performance' at the undergraduate level?
2) Does the use of the tool, without prior training or tutorials, significantly impact learning outcomes?

A. Planning Learning Evaluation

To address these questions, it is essential to observe undergraduate students' interactions with the tool and to assess their learning needs. Following this, students should engage in hands-on exercises using CacheLab, enabling a structured evaluation of their learning experience.

The study was structured into two complementary phases: a theoretical introduction and a practical activity. The first phase consisted of two sessions (totaling four hours) focused on key concepts of cache memory, design strategies, and performance evaluation. The second phase involved hands-on experimental exercises, which were individually performed by the students without any teacher intervention or the use of tutorials. This approach aimed to assess the students' ability to independently explore the tool and apply the theoretical knowledge acquired.

The target audience consisted of fifty-three (53) fourth- and fifth-semester students from the Computer Science and Computer Engineering programs, offered by the Federal University of Mato Grosso do Sul, Brazil. The pedagogical curricula of these two programs guarantee that the students involved in the study had already successfully completed a previous course addressing cache memory.

Student learning was assessed through a theoretical questionnaire and an experimental report. To evaluate the tool's impact on students' conceptual knowledge, the same conceptual assessment form was applied twice to each student: once before and once after the simulator use. The questionnaire was published in the university official virtual learning environment, using multiple-choice questions, concept association, and open questions to address 25 essential concepts about caches, described in Table I. The questionnaire also included two questions about students' perceived knowledge of caches, focusing on concepts, design, and performance. As all participants were Computer Architecture students from the same university and term, we expected similar levels of prior knowledge.

TABLE I: Conceptual Questionnaire

Topic	Concepts
Cache Design	Temporal and spatial locality concepts and examples, blocks, words per block, associativity, blocks per set, replacement policy, block fields, capacity, byte offset, block offset, tag;
Cache Performance	Hit rate, miss rate, average memory access, hit_time, miss_penalty.
Students' Perceived	Questions assessing the students' self-perceived knowledge about caches design and performance.

B. Planning User Experience Evaluation

The tool's user experience and perception were evaluated using a survey modeled on the Technology Acceptance Model (TAM), introduced by Fred Davis in 1986 [14]. TAM provides a well-established theoretical framework for assessing and predicting user acceptance of new technologies. Initially developed as a practical tool to evaluate user adoption in the early stages of system design, TAM has become a cornerstone in technology acceptance research. Central to the model are two key determinants: Perceived Usefulness (PU) and Perceived Ease of Use (PEOU). Perceived Usefulness is defined as the degree to which an individual believes that using a particular system would improve their performance, while Perceived Ease of Use refers to the degree to which a person believes that using the system would be free of effort. By identifying and quantifying these constructs, TAM offers a systematic basis for refining system design and enhancing adoption rates. Notably, the model highlights the robustness of PU, showing that even non-interactive evaluations can reliably predict user acceptance. This feature enables the integration of usability testing and user feedback early in the development process, thereby reducing the likelihood of deploying systems that do not align with user expectations.

An online questionnaire was developed to collect data on the user experience for this part of the research. The survey was structured into four sections:

1) **Introduction and Consent:** a brief overview of the study, including its objectives and methodology. Participants were also required to give explicit consent for using their responses in the research.
2) **Profile:** questions regarding their demographic and academic background, such as age, gender, degree program, prior formal instruction in computer architecture, and prior formal instruction in design space exploration.
3) **Ease of Use:** statements about the user experience concerning the tool's ease of use. Participants were asked to indicate their level of agreement with each statement. An open-ended question at the end of the section invited participants to share their thoughts on the PEOU freely.
4) **Usefulness:** statements regarding the PU of the tool and its impact on the learning experience. Participants were again asked to indicate their level of agreement. Two open-ended questions concluded the section, allowing participants to express their views on the tool's usefulness and perceived impact on their learning.

In both the Ease of Use and Usefulness sections, participants indicated their level of agreement using a five-point Likert scale: **1-Completely Disagree; 2-Partially Disagree; 3-Neutral; 4-Partially Agree; 5-Completely Agree**.

These sections also included open-ended questions addressing usability, usefulness, and learning. The responses to these questions were analyzed and categorized according to a five-level scale based on the overall tone of the feedback:

Negative: The participant's narrative mentioned only drawbacks, difficulties, and possible improvements, with no positive remarks.

Mostly Negative: The participant included some positive points or praise, but the focus remained on criticisms and suggested improvements.

Neutral: The participant's feedback offered a balanced view, highlighting both strengths and weaknesses of the tool.

Mostly Positive: The feedback emphasized positive aspects and praise, with some mention of minor issues.

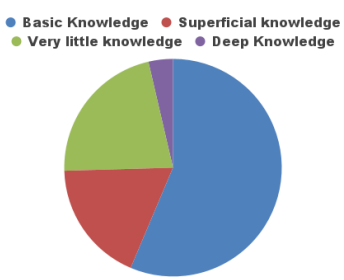
Positive: The participant highlighted only positive points and praise.

V. Results and Discussion

This section summarizes the data collected and analyzed from the case study and discusses the results. We analyze the data from the students' evaluation and the technology acceptance survey. Fifty-three (53) students submitted valid responses to the conceptual questionnaire (before and after using CacheLab) and completed the technology acceptance form.

A. Results on Learning Evaluation

The preliminary theoretical questionnaire was designed to assess their prior knowledge of cache concepts before interacting with the CacheLab. Figure 4a shows the students' self-assessed knowledge regarding cache design concepts. Most students reported having a basic understanding, followed by a notable portion indicating superficial knowledge. This distribution suggests a relatively modest level of confidence among students in their understanding of cache design, highlighting the importance of reinforcing these concepts through practical learning tools, such as CacheLab. Figure 4b presents the students' perceived knowledge of cache performance. Similarly to the previous chart, the majority reported basic knowledge. Still, there was a slight shift: a larger percentage of students rated themselves as having very little knowledge, and fewer reported having either a superficial or deep understanding compared to the design dimension. This information indicates that cache performance is an area where students feel even less confident, potentially due to the complexity involved in analyzing performance metrics and optimization strategies.



(a)

(b)

Fig. 4: Students' perceived knowledge about cache design (a), and cache performance (b) before using the CacheLab.

After answering the first questionnaire, the same students were instructed to complete fifteen experimental exercises aimed at fostering familiarity with the simulator, without receiving any prior training or instructional material regarding its use. After completing the exercises, the students were invited to retake the conceptual questionnaire.

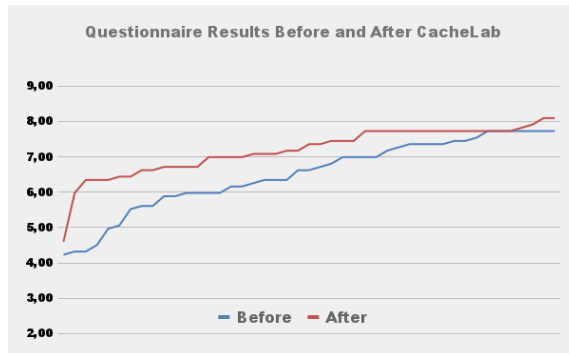


Fig. 5: Comparison of students' scores on the conceptual questionnaire before and after completing experimental exercises with the CacheLab simulator

The graph in Figure 5 compares students' performance on the conceptual questionnaire administered before and after completing the experimental exercises using the CacheLab simulator. Each line represents the distribution of individual scores, sorted in ascending order. A clear upward shift in performance is observed following the use of CacheLab. The post-intervention results (red line) show a consistent improvement across the entire score range. Furthermore, the post-intervention curve consistently lies above the pre-intervention curve, highlighting a positive learning effect. The average score increased by approximately 9%, indicating that the hands-on activities had a positive impact on students' conceptual understanding. The modest but steady increase in scores suggests that CacheLab was effective in reinforcing theoretical concepts through hands-on experimentation. This trend supports the hypothesis that practical engagement with cache-related mechanisms through simulation can enhance conceptual understanding, even in the absence of direct instruction or tutorials.

It is worth noting that there was no direct monitoring of the students during the execution of the activities using CacheLab. The students only received the link to the tool and the instructions for the experiments. Our interpretation of the results, especially the students' feedback (observations and suggestions), is that the learning experience could have been significantly enhanced in a controlled environment with the support of teaching assistants (for example, during the delivery of a short course or a hands-on class). However, our goal in this study was to avoid any interference in the learning process, so that any benefits observed could be directly attributed to the use of the simulator.

A statistical analysis was conducted to compare the grade distributions before and after the use of the simulator. The normality of the distributions was initially assessed using the Shapiro-Wilk test [15] and the D'Agostino-Pearson test [16].

The results indicated that the grades obtained before and after, as well as the difference between them, did not follow a normal distribution. Therefore, the non-parametric Wilcoxon signed-rank test [17] was applied, which is suitable for paired samples when the normality assumption is violated.

The test revealed a statistically significant difference between the distributions (W = 112.5; p = 0.00031), allowing the rejection of the null hypothesis at a 5% significance level. The analysis revealed that the median post-test score was higher than the median pre-test score, indicating a significant improvement in students' performance. The calculated effect size (r = 0.3503) corresponds to a medium effect, reinforcing the potential of the tool as a practical learning support resource.

B. Results on Experience Evaluation

Usability was assessed through four multiple-choice questions and one open-ended question, in which users were invited to freely describe their experience with the tool, focusing on its ease of use. A summary of the responses to the multiple-choice questions is presented in Table II.

TABLE II: PEOU Results

Question	1	2	3	4	5
Using the tool was easy	0.0%	5.7%	15.1%	45.3%	34.0%
I was able to use the tool as I wanted	0.0%	11.3%	5.7%	37.7%	45.3%
I understood what was happening while I was using the tool	1.9%	13.2%	13.2%	43.4%	28.3%
I was able to easily carry out a project and a performance evaluation activity using the tool	1.9%	5.7%	17.0%	34.0%	41.5%

TABLE III: PU Results

Question	1	2	3	4	5
The tool was useful for understanding the behavior and evaluating the performance of caches	0.0%	1.9%	7.5%	45.3%	45.3%
Using the tool helped me understand how to implement a new cache design and the impact of parameters on system performance	0.0%	0.0%	17.0%	47.2%	35.8%
Using the tool helped me understand what some of the key parameters of memory system design are, as well as their interrelationships	0.0%	0.0%	15.1%	41.5%	43.4%
Using the tool helped me understand the impacts of cache design parameters on the memory system	0.0%	1.9%	11.3%	30.2%	56.6%

The data reveal a generally favorable perception across all items, with the majority of students selecting higher values on the Likert scale (4 or 5). *"Using the tool was easy"* received predominantly positive responses, with 79.3% of students selecting 4 or 5, indicating that most participants found the tool intuitive and accessible. *"I was able to use the tool as I wanted"* shows an even stronger endorsement, with 83.0% of responses falling in the top two categories. This insight suggests that the simulator allowed students the autonomy and flexibility to interact with it as intended.

For the statement *"I understood what was happening while I was using the tool"*, the majority again responded positively (71.7% selected 4 or 5). However, a small proportion (13.2%) indicated moderate difficulty in interpreting the tool's behavior (option 2). Finally, *"I was able to easily carry out a project and cache performance evaluation activity using the tool"* yielded 75.5% agreement in the highest categories, showing that students not only found the tool usable but also effective for completing practical tasks.

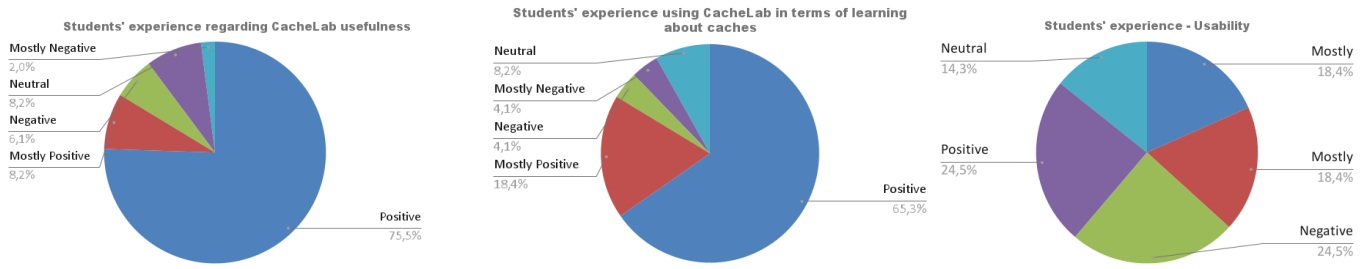
Usefulness was measured similarly, with four multiple-choice questions and two open-ended questions, one of which was aimed at assessing the usefulness of the tool and the other at evaluating the experience in terms of learning about cache design and performance. A summary of the data from the multiple-choice question is shown in Table III and the open questions in Figure 6.

The results demonstrate a strong consensus among students regarding the usefulness of the CacheLab simulator in supporting their understanding of cache-related concepts. Specifically, students indicated that the tool helped them understand cache behavior and performance evaluation, as well as grasp key memory system design parameters and their interrelationships.

As mentioned before, we also encourage students to share their experience in two open-ended questions about usefulness and one open-ended question about usability. The results are presented in Figure 6.

Figure 6a captures students' perceptions of CacheLab's overall usefulness. An overwhelming majority (75.5%) rated the tool as positive in terms of usefulness, and another 8.2% considered it mostly positive. Negative responses accounted for less than 10% in total, with only 2.0% reporting a mostly negative experience. Figure 6b evaluates students' experiences with CacheLab specifically in the context of learning cache-related concepts. A clear majority (65.3%) rated the experience as positive, followed by 18.4% who indicated a mostly positive impression. Only a small minority (8.2%) expressed negative or mostly negative opinions, while 8.2% remained neutral.

The students' perceptions regarding the usability of the CacheLab simulator are presented in Figure 6c. The responses are relatively balanced, with 24.5% rating their experience as positive and an equal proportion (24.5%) rating it as negative. This insight suggests that while a segment of students found the tool user-friendly, another significant portion encountered difficulties. The mixed results may reflect a lack of initial guidance or an interface that could benefit from improvements in intuitiveness and accessibility.



(a) (b) (c)

Fig. 6: Students' perceptions regarding usefulness and usability, categorized according to a five-level scale based on the overall tone of the feedback.

VI. Conclusion

This paper introduces CacheLab, a functional cache memory simulator designed to support the teaching of Computer Architecture and related disciplines in Computer Science courses at the Federal University of Mato Grosso do Sul in Brazil.

These results indicate that, from the students' perspective, CacheLab demonstrates strong characteristics of usefulness and usability, aligning with the principles of user-centered design in educational technologies. However, the presence of responses in the lower and middle categories across items also suggests that a subset of users encountered barriers, which may point to areas for interface refinement or the need for minimal onboarding resources. Despite the usability challenges, the findings strongly support the notion that CacheLab is a valuable educational tool, contributing meaningfully to students' understanding, despite the usability challenges identified in the survey. The tool served as a meaningful pedagogical aid, bridging the gap between theoretical content and its practical implications in computer architecture.

A distinguishing feature of the CacheLab tool is its ability to execute memory traces collected from real-world applications. Furthermore, the CacheLab repository offers a tool for generating new memory traces, allowing users to create customized datasets tailored to their experiments. Although this functionality was not explored in the case study presented in this paper, it is already planned for future work. Additionally, all suggestions provided by the students will be systematically compiled and considered for the development of new features in future versions of the tool.

The CacheLab is publicly available at and the Parser to address novel memory traces is available at https://cache-parser.hugo.dev.br/. Public repositories are available in https://github.com/lscad-facom-ufms/CacheLab.

VII. References

[1] David A Patterson and John L Hennessy. *Computer organization and design ARM edition: the hardware software interface*. Morgan kaufmann, 2016.

[2] John L Hennessy and David A Patterson. *Computer architecture: a quantitative approach*. Elsevier, 2011.

[3] LeandroGabriel.net. Beto: o simulador de memória cache. https://simuladorcache.leandrogabriel.net, 2022.

[4] Arthur Wuterich. A configurable cache simulator. https://rivoire.cs.sonoma.edu/cs351/resources.html.

[5] Swadhesh Kumar and PK Singh. An overview of modern cache memory and performance analysis of replacement policies. In *2016 IEEE International Conference on Engineering and Technology (ICETECH)*, pages 210–214. IEEE, 2016.

[6] University Of Washington. Cache simulator. https://courses.cs.washington.edu/courses/cse351/cachesim/.

[7] SideChannelMarvels. Tracer: Set of dynamic binary instrumentation and visualization tools for execution traces. https://github.com/SideChannelMarvels/Tracer, 2024.

[8] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. *ACM Sigplan notices*, 42(6):89–100, 2007.

[9] Omitted. Tracergrind-parser: Tracergrind texttrace dump parser. https://github.com/HugoJF/cache-simulator-tracergrind-parser, 2024.

[10] Evan You. Vite: Next generation frontend tooling. it's fast! https://vite.dev.

[11] Anthony Fu Vladimir and Ari Perkkiö. Vitest: Next generation testing framework powered by vite. https://vitest.dev.

[12] AntGroup. Ant design: An enterprise-class ui design language and react ui library. https://ant.design.

[13] Adam Wathan. Tailwindcss: A utility-first css framework for rapid ui development. https://tailwindcss.com.

[14] Fred D Davis. Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS quarterly*, pages 319–340, 1989.

[15] S. S. Shapiro and M. B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3-4):591–611, 1965.

[16] Ralph D'Agostino and E. S. Pearson. Tests for departure from normality. empirical results for the distributions of b2 and b1. *Biometrika*, 60(3):613–622, 1973.

[17] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.